

Apuntes: apuntores (pointers) en C

Contexto: estos apuntes surgen de entender por qué `uart_write_bytes` y `uart_read_bytes` piden un apuntador en vez del valor directo, en la práctica `1_Bytes`. Sirven como repaso general de apuntores en C, no solo para UART.

1. Una variable es una caja con dirección

Toda variable vive en dos lugares a la vez: tiene un **contenido** (el valor que guarda) y una **dirección** (el lugar físico de la memoria donde vive esa caja).

```
uint8_t valor = 128;
```

- `valor` → el contenido de la caja: `128`.
- `&valor` → la dirección de la caja: algo como `0x3FFB2A10`.

El operador `&` significa “la dirección de”. No te da lo que hay adentro, te dice dónde está.

2. El operador `*` tiene dos usos distintos

Esto es lo que más confunde al inicio, porque el mismo símbolo significa dos cosas diferentes según el contexto:

Al declarar una variable, `*` dice “esta variable no guarda un valor normal, guarda una dirección”:

```
uint8_t *p; // p es un apuntador: va a guardar la direccion de un uint8_t
```

Al usar una variable ya declarada, `*` significa “ve a esa dirección y tráeme (o escribe) lo que hay ahí” — esto se llama *desreferenciar*:

```
p = &valor;           // p ahora guarda la direccion de "valor"
printf("%d", *p);     // *p = "lo que hay en esa direccion" = 128
*p = 200;             // escribe 200 directamente en la direccion que p guarda
                     // (valor ahora vale 200, aunque no se toco "valor" por su nombre)
```

3. ¿Por qué una función pediría un apuntador en vez del valor?

Hay dos razones distintas, y cubren prácticamente todos los casos:

Razón A: la función necesita modificar tu variable

En C, cuando pasas una variable normal a una función, la función recibe una **copia**. Cambiar la copia no afecta a la variable original:

```
void incrementar(int x) {
    x = x + 1;    // solo cambia la copia
}

int numero = 5;
incrementar(numero);
// numero sigue siendo 5
```

Si en cambio le pasas la dirección, la función puede ir directo a la caja original y modificarla:

```
void incrementar(int *x) {
    *x = *x + 1;    // ve a la direccion y cambia lo que hay ahi
}

int numero = 5;
incrementar(&numero);
// ahora numero es 6
```

Una función en C solo puede regresar **un** valor con `return`. Si necesita darte más de un resultado, o modificar algo que ya existe, la única forma es que le pases la dirección para que escriba ahí directamente.

Ejemplo real: `uart_read_bytes(UART_PUERTO, &valor_recibido, 1, 0)`. El `return` de la función ya se usa para decir cuántos bytes llegaron; el dato en sí solo puede dártelo escribiéndolo en la dirección de `valor_recibido`.

Razón B: el dato es un bloque de memoria, no un solo valor

Un arreglo, un buffer, un `struct` — en C no se pueden “pasar completos” como se pasa un `int`. Un bloque de datos siempre se describe como **dirección de inicio + tamaño**, y esa es la única forma que tiene C de manejarlo.

Ejemplo real: `uart_write_bytes(UART_PUERTO, &valor_enviado, 1)`. Aunque aquí mandamos un solo byte, la función está diseñada de forma genérica para mandar bloques de cualquier tamaño — por eso pide dirección + cantidad de bytes, no el valor.

4. ¿Por qué no hacía falta el `(const char *)`?

El código original tenía `uart_write_bytes(UART_PUERTO, (const char *)&valor_enviado, 1)`. El cast sobra, y esta es la justificación completa:

1. La función en realidad pide `const void *src` — `void *` es el “puntero genérico” de C, el que se usa en funciones de manejo de memoria (`memcpy`, `malloc`, etc.) precisamente porque no le importa de qué tipo es el dato.
2. El estándar de C permite convertir **cualquier** puntero a `void *` de forma implícita, sin cast — es una regla explícita del lenguaje, no una relajación del compilador. Por eso puedes pasarle un `int *`, un `uint8_t *`, lo que sea.
3. Agregar `const` también se permite de forma implícita (ir de “menos restringido” a “más restringido” siempre es seguro). Lo que sí exigiría cast es el caso contrario: quitarle el `const` a algo.

Entonces `uint8_t *` $\rightarrow$ `const void *` es una conversión completamente legal sin forzar nada. Por eso `uart_write_bytes(UART_PUERTO, &valor_enviado, 1)` compila limpio, sin cast.

(Nota aparte: `char` y `uint8_t` en la práctica son el mismo tamaño — `uint8_t` es un alias de `unsigned char`. `char` aparece tan seguido en este tipo de funciones por costumbre histórica: antes de que existiera `uint8_t` en `<stdint.h>`, `char` era la forma estándar de decir “un byte crudo” en C.)

5. Arreglos: `mensaje` VS `&mensaje`

Con una variable individual (`uint8_t`, `int`, `float`) siempre hace falta `&` para conseguir su dirección. Con un **arreglo**, la regla cambia.

```
char mensaje[] = "Hola\r\n";
uart_write_bytes(UART_PUERTO, mensaje, strlen(mensaje));
```

Aquí `mensaje` va sin `&`. La razón es una regla especial de C llamada **decaimiento de arreglo** (*array decay*): casi cualquier vez que usas el nombre de un arreglo en una expresión (incluido pasarlo como argumento de una función), C lo convierte automáticamente en un puntero a su primer elemento. Es decir, `mensaje` ya es, por sí solo, la dirección de `mensaje[0]`.

¿Y si igual escribes `&mensaje`?

Apunta al mismo lugar en memoria — la dirección numérica es idéntica — pero el **tipo de dato es distinto**:

- `mensaje` decae a `char *`: “aquí hay **un char**”. El compilador lo interpreta elemento por elemento.
- `&mensaje` es de tipo `char (*)[7]`: “aquí hay **un arreglo completo de 7 chars**”. El compilador lo interpreta como un bloque de una sola pieza.

Mismo valor de dirección, pero uno describe “un elemento” y el otro describe “el arreglo completo” — son tipos de dato distintos aunque coincidan en el número de la dirección.

Por qué la notación `char (*)[7]` no es “un arreglo de 7 direcciones”

Es fácil confundirla con `char *[7]` (“arreglo de 7 punteros a char”), pero son lo opuesto:

```
char (*p)[7];    // p es UN puntero, apunta a un arreglo de 7 chars
char *p[7];      // p es UN arreglo de 7 punteros a char
```

La regla de precedencia en C: `[]` (arreglo) se pega al nombre antes que el `*`. Sin paréntesis, `*p[7]` se lee “`p` es `[7]` de (`*` de `char`)” = arreglo de 7 punteros. Para que sea al revés — un solo puntero que apunta a un bloque completo — hay que agrupar `*p` con paréntesis: `(*p)[7]`.

La diferencia importa para la aritmética de punteros

El tipo de un puntero no es solo una dirección: también define **cuánto avanza** cuando le sumas 1.

- `char *p` (apunta a un elemento): `p + 1` avanza **1 byte** — al siguiente char.
- `char (*p)[7]` (apunta a un bloque de 7): `p + 1` avanza **7 bytes** — al siguiente bloque completo de 7 chars.

Por eso `uart_write_bytes` exige específicamente el tipo “puntero a un elemento” (`char *` , o cualquier cosa convertible a `void *` con esa semántica): necesita poder copiar byte por byte a partir de esa dirección, y un `char (*) [7]` no le da esa garantía de tipo, aunque la dirección de arranque sea la misma.

Resumen

Caso	Cómo se pasa	Tipo resultante	Por qué
Variable individual (<code>uint8_t</code> , <code>int</code> ...)	<code>&variable</code>	puntero a un elemento	hay que forzar la dirección con <code>&</code>
Arreglo (<code>char[]</code> , buffer...)	<code>nombre_arreglo</code>	puntero a un elemento (decae)	el arreglo ya decae solo a la dirección de su primer elemento
Arreglo, forzando la dirección completa	<code>&nombre_arreglo</code>	puntero al bloque completo	tipo distinto: describe todo el arreglo como una unidad, no elemento por elemento